

112 學年第 1 學期

輔仁大學

人工智慧與資訊安全學士學位學程

劉昱甫

# 期末報告

Threat Hunting Essential

Attack Recognition Techniques

Part 2

|                                                       |    |
|-------------------------------------------------------|----|
| 第一章、 環境設置 .....                                       | 3  |
| 第一節、 整體架構 .....                                       | 3  |
| 第二節、 opnsense 中 wazuh agent 以及 IDS (suricata) 設定..... | 6  |
| 第二章、 成果展示 .....                                       | 12 |
| 第一節、 使用 wazuh mcp.....                                | 12 |
| 第二節、 將 pfSense 的資料有效傳送到 wazuh 上進行偵測與回應.....           | 15 |
| 第三節、 ET-BERT 使用體驗.....                                | 17 |
| 第三章、 總結及參考資料 .....                                    | 18 |
| 第一節、 總結 .....                                         | 18 |
| 第二節、 參考資料 .....                                       | 19 |

## 第一章、環境設置

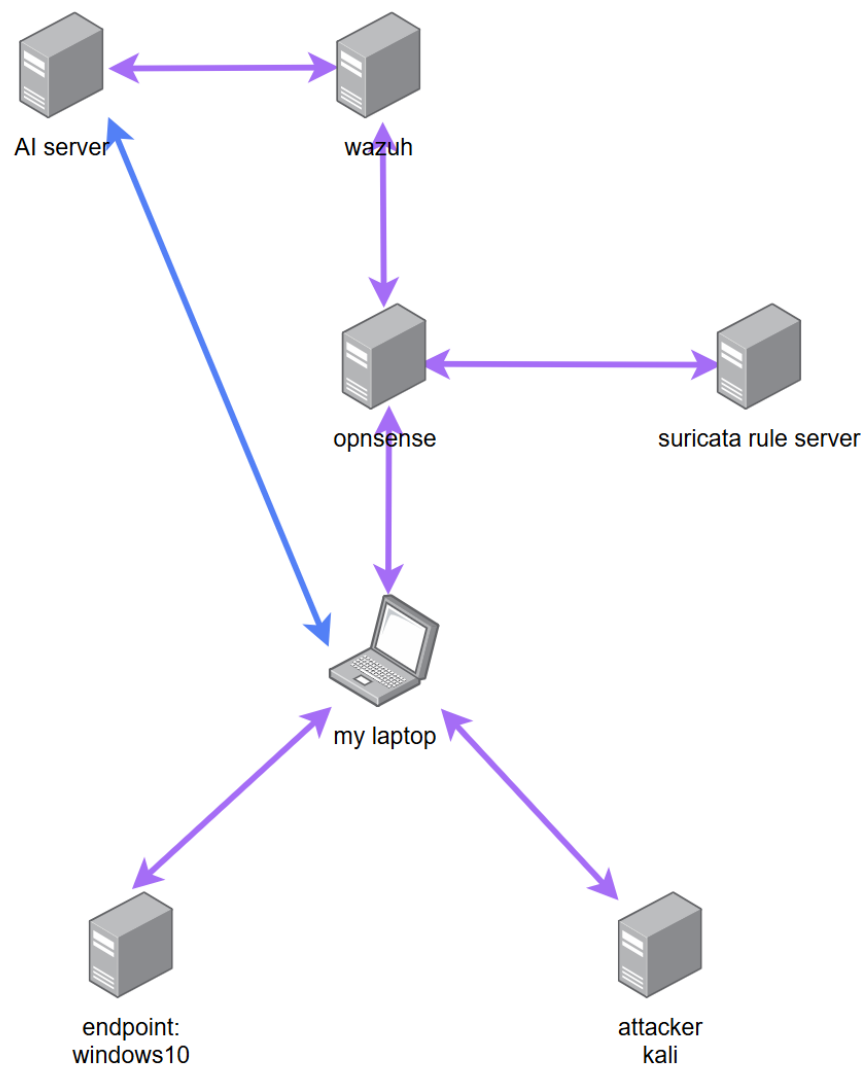
### 第一節、整體架構

圖一中為作業第一題和第二題會用到的設備，有

- 伺服器(pve)
  - wazuh server
  - opnsense
  - suricata rule server
  - VPN server (使用 wireguard)
- 在本地端機器上(我的筆電)
  - windows 10 vm
  - kali vm
- AI server(我家裡的 windows 11 桌機)

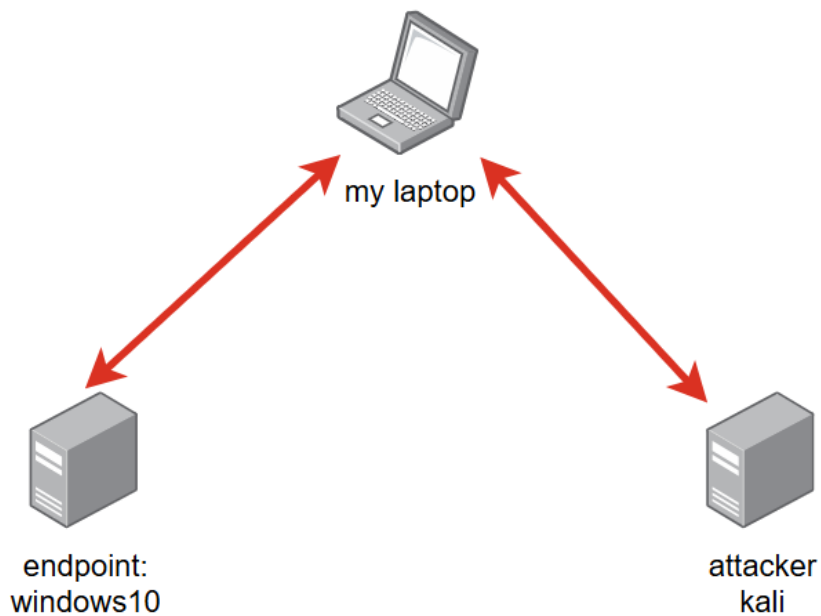
我為了分散每台機器(包括 pve 伺服器、到處跑的筆電以及家裡的桌機)的運行

壓力，所以才會這樣設計。



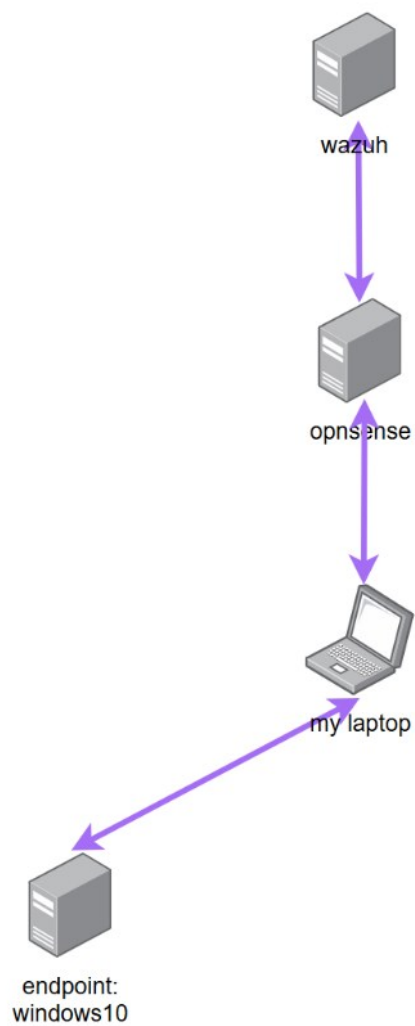
圖一、作業第一題和第二題整體架構

如圖二所示，我把 windows 虛擬機(我們假定要被攻擊的目標，下簡稱端點 windows)以及 kali(我們假定要做攻擊的機器，下簡稱 kali)設定在同一個網路環境下，kali 可以直接訪問到端點 windows。



圖二、端點 windows 虛擬機和攻擊者 kali 的架構

同時如圖三所示，端點 windows 可將日誌(log)傳到 wazuh server 上，並且所有流量都會通過 opnsense(這個架在 pve 上的設備有將很多功能集成在上面，有防火牆、路由、IDS 等功能，同時這也是 pfsense 的另一個 distro，我以下簡稱 opnsense) 這個設備，最後才會將所有日誌(包括 windows 上的以及 opnsense 的)



圖三、端點 windows 10 和 wazuh 的架構

## 第二節、opnsense 中 wazuh agent 以及 IDS (suricata) 設定

由於 opnsense 社群有設計專門給其設計對應的 wazuh agent(再次感嘆開源社群的強大)之插件(plugin)所以我們直接到 opnsense

System>Firmware>Plugins 中就可以找到 os-wazuh-agent 的插件。

然後接著就是在 Services>Wazuh Agent>Settings 中調設定，除了在 Applications 中的 Firewall 和 Suricata(IDS)在我們這次實作中會用到之外，其餘設定跟把 wazuh agent 佈署在其他端點中差不多。詳細可看圖四

opnsense 內建 Intrusion Detection 的功能，並且其使用 suricata。如圖五所示，我設定只有兩個網路介面，分別是模擬內網的 VPN 以及會連到網際網路的 WAN，特徵偵測(Pattern Matcher)我將其設定為 Hyperscan。

註: 在開啟 IDS 前我有先到 suricata 的 Interfaces>Settings 中設定 Hardware CRC、Hardware TSO、Hardware LRO 以及 VLAN Hardware Filtering 關閉

不過由於沒辦法直接在 opnsense 上面直接設定 IDS 的規則，所以用到自架的 suricata rule server，我在上面寫 opnsense 使用 rule server 的設定 (custom.xml)以及 IDS 的規則(custom.rules)，詳見圖六、圖七。其中我用 scp 指令將 custom.xml 放入 opnsense 裡面，然後 opnsense 的 IDS 就會知道要去 suricata rule server 上更新我們自定義的 IDS 規則，對此我是用 python 打開 http server 去實現讓 opnsense 更新 IDS 規則。詳見圖八

這樣，把 opnsense 中 Intrusion Detection 的服務重啟後，並更新規則後，就可以成功使用我們自訂的內容。具體如圖九、圖十。

見圖十一，到 Services>Intrusion Detection>Administration>Rules(或是 Policy 中的 Rule adjustments)，我們打開任意的規則細節後，可以自由選擇當偵測到該細節時是要選擇警告然後放行，還是直接將該流量擋掉。

## Services: Wazuh Agent: Settings

Settings

advanced mode

General Settings

Enable

☒

Manager hostname

10.2.26.23

Agent hostname

opnsense-wazuh-go

Protocol

TCP

Manager port

1514

Applications

audit (audit), configd (configd.py), dhcpd (dhcpd)

Clear All

Select All

Intrusion detection events

☒

Logcollector remote commands

☒

Debug

first level of debug

Active response

Enable

☒

Firewall command ignore

None

Wazuh remote commands

☒

Enrollment

Password

Enrollment port

1515

Policy monitoring and anomaly detection

Enable

☒

System inventory

Enable

☒

File integrity monitoring

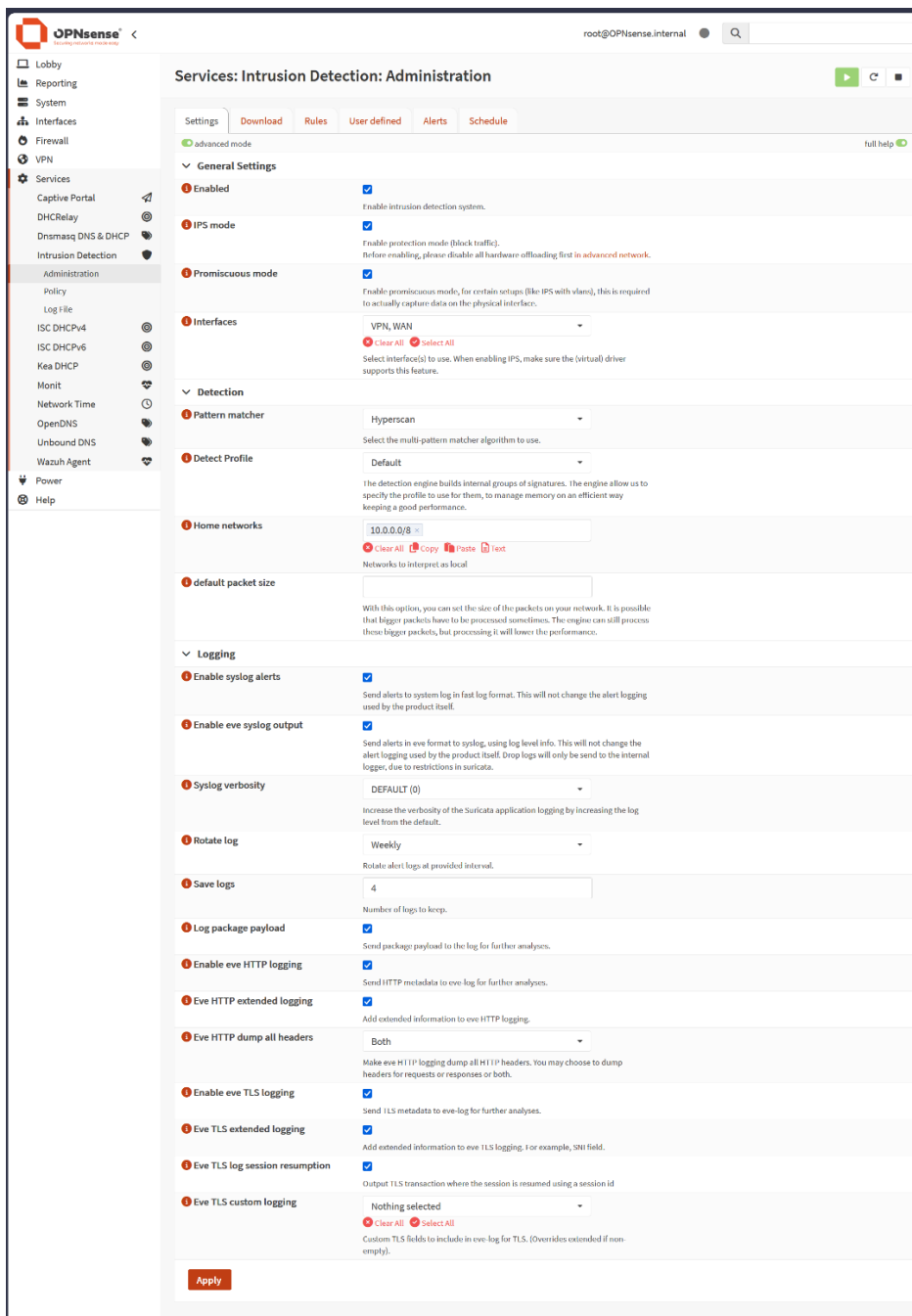
Enable

☒

OPNsense (c) 2014-2025 Deciso B.V.

圖四、opnsense wazuh agent 設定參考





圖五、opnsense IDS 的一般設定參考

```
<?xml version="1.0"?>
<ruleset documentation_url="http://docs.opnsense.org/">
  <location url="http://10.2.26.25/" prefix="custom" />
  <files>
    <file description="custom rules">custom.rules</file>
    <file description="custom" url="inline::rules/custom.rules">custom.rules</file>
  </files>
</ruleset>
```

圖六、custom.xml 設定

```
# CUSTOM RULESET FOR TEST
# Nmap / SSH / Rustdesk
#
# ——— NMAP SCANS ———
alert tcp any any → any any (flags:S; msg:"CUSTOM: Nmap SYN scan detected"; sid:900001; rev:1;)
alert tcp any any → any any (flags:0; msg:"CUSTOM: Nmap NULL scan detected"; sid:900002; rev:1;)
alert tcp any any → any any (flags:F; msg:"CUSTOM: Nmap FIN scan detected"; sid:900003; rev:1;)
alert tcp any any → any any (flags:FAUP; msg:"CUSTOM: Nmap Xmas scan detected"; sid:900004; rev:1;)
# ——— SSH ———
alert tcp any any → any 22 (msg:"CUSTOM: SSH connection attempt"; sid:900010; rev:1;)
alert tcp any any → any 22 (flags:S; threshold:type both,track by_src,count 5,seconds 10; msg:"CUSTOM: SSH scanning behavior detected"; sid:900011; rev:1;)
alert tcp any any → any 22 (threshold:type threshold,track by_src,count 15,seconds 60; msg:"CUSTOM: SSH brute force attempt"; sid:900012; rev:1;)
# ——— RUSTDESK ———
alert tcp any any → any 21116 (msg:"CUSTOM: Rustdesk Relay connection"; sid:900020; rev:1;)
alert udp any any → any 21116 (msg:"CUSTOM: Rustdesk Relay UDP traffic"; sid:900021; rev:1;)
alert tcp any any → any 21115 (msg:"CUSTOM: Rustdesk ID Server traffic"; sid:900022; rev:1;)
alert udp any any → any any (dsize:>50; threshold:type both,track by_src,count 100,seconds 5; msg:"CUSTOM: Rustdesk P2P-like UDP burst"; sid:900023; rev:1;)
```

圖七、custom.rules 設定

```
nisra@nisra:~$ sudo python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.100.0.101 - - [28/Nov/2025 14:31:37] "GET / HTTP/1.1" 200 -
10.2.26.24 - - [28/Nov/2025 14:32:07] "GET /custom.rules HTTP/1.1" 200 -
```

圖八、http server 使用 python，圖中可以看到有兩個 IP 嘗試存取此設備，一個是我的電腦，另外一個則是 opnsense

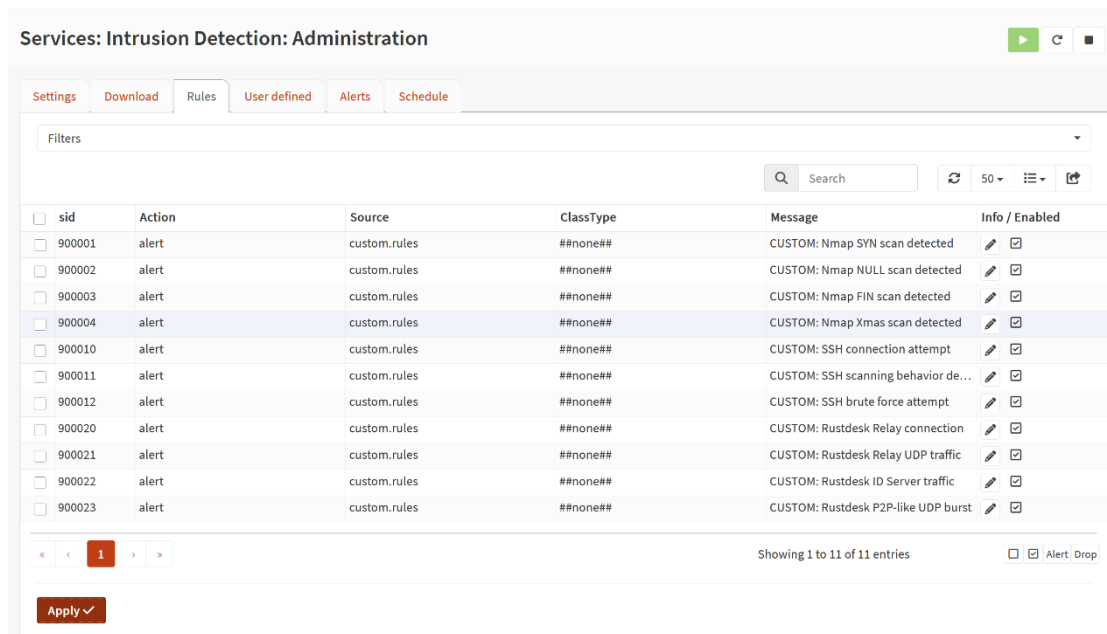
Services: Intrusion Detection: Administration

Settings Download Rules User defined Alerts Schedule

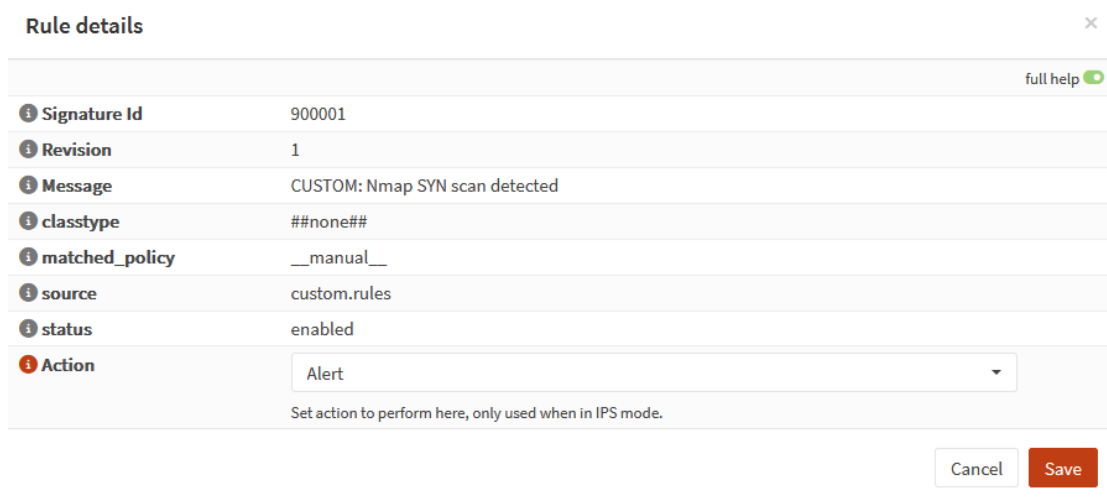
Rulesets Enable selected Disable selected Search

| Description                                                 | Last updated     | Enabled | Edit |
|-------------------------------------------------------------|------------------|---------|------|
| <input type="checkbox"/> abuse.ch/Feodo Tracker             | not installed    | x       |      |
| <input type="checkbox"/> abuse.ch/SSL Fingerprint Blacklist | not installed    | x       |      |
| <input type="checkbox"/> abuse.ch/SSL IP Blacklist          | not installed    | x       |      |
| <input type="checkbox"/> abuse.ch/ThreatFox                 | not installed    | x       |      |
| <input type="checkbox"/> abuse.ch/URLhaus                   | not installed    | x       |      |
| <input type="checkbox"/> custom/custom rules                | 2025/11/28 14:32 | ✓       |      |

圖九、可以看到 opnsense 有讀取到 custom.xml 的內容了，並且成功寫入進去



圖十、可以看到匯入到 opnsense 的 IDS 規則已經可供我們使用



圖十一、我們打開任意的規則細節後，可以自由選擇當偵測到該  
細節時是要選擇警告然後放行，還是直接將該流量擋掉

於是這樣我們就正式設定完成 opnsense 上防火牆和 IDS 與 wazuh 的交互

## 第二章、成果展示

### 第一節、使用 wazuh mcp

如若 MCP server 和 AI server 放在一起，這會遇到，wazuh indexer 只聽 localhost:9200 port。所以需要先在 wazuh server 中修改 wazuh indexer 的設定檔案，讓 server 不會只聽 localhost。詳見圖十二。

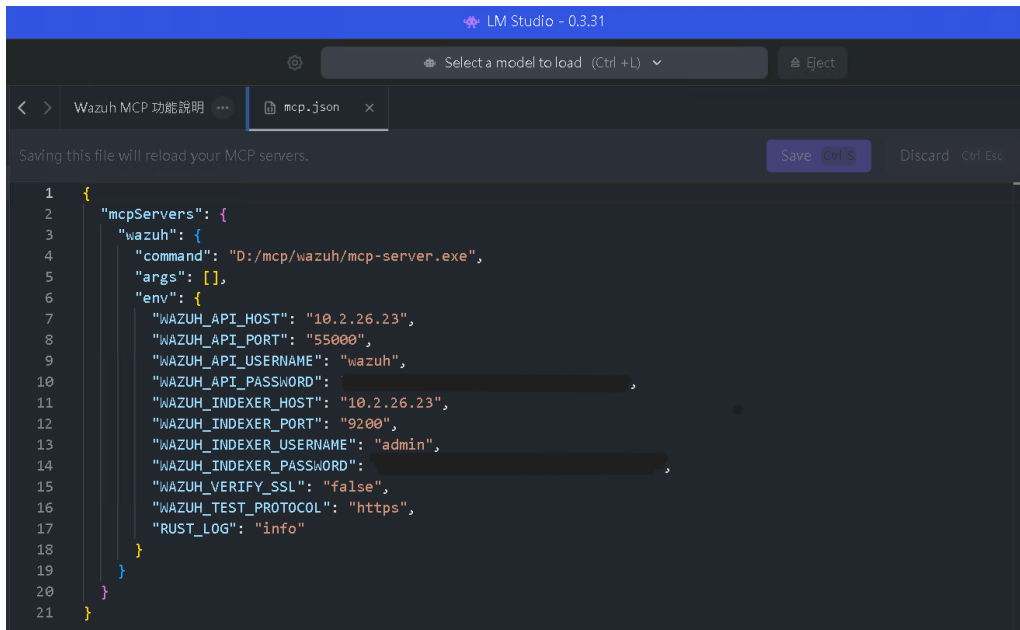
接著在 AI server 上設定 mcp server，由於我希望使用本地的大語言模型 (LLM) 去跑(當然在 cloude desktop 上我也有跑過)，所以我還要修改軟體 (LM studio)裡的 MCP.json 即可。

```
network.host: "0.0.0.0"
node.name: "node-1"
cluster.initial_master_nodes:
- "node-1"
cluster.name: "wazuh-cluster"

node.max_local_storage_nodes: "3"
path.data: /var/lib/wazuh-indexer
path.logs: /var/log/wazuh-indexer

plugins.security.ssl.http.pemcert_filepath: /etc/wazuh-indexer/certs/wazuh-indexer.pem
plugins.security.ssl.http.pemkey_filepath: /etc/wazuh-indexer/certs/wazuh-indexer-key.pem
plugins.security.ssl.http.pemtrustedcas_filepath: /etc/wazuh-indexer/certs/root-ca.pem
plugins.security.ssl.transport.pemcert_filepath: /etc/wazuh-indexer/certs/wazuh-indexer.pem
plugins.security.ssl.transport.pemkey_filepath: /etc/wazuh-indexer/certs/wazuh-indexer-key.pem
plugins.security.ssl.transport.pemtrustedcas_filepath: /etc/wazuh-indexer/certs/root-ca.pem
plugins.security.ssl.http.enabled: true
plugins.security.ssl.transport.enforce_hostname_verification: false
plugins.security.ssl.transport.resolve_hostname: false
plugins.security.ssl.http.enabled_ciphers:
- "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256"
- "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384"
- "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256"
"/etc/wazuh-indexer/opensearch.yml" 41L, 2150B 10,0-1 Top
```

圖十二、/etc/wazuh-indexer/opensearch.yml 的設定



```
1 {
2   "mcpServers": {
3     "wazuh": {
4       "command": "D:/mcp/wazuh/mcp-server.exe",
5       "args": [],
6       "env": {
7         "WAZUH_API_HOST": "10.2.26.23",
8         "WAZUH_API_PORT": "55000",
9         "WAZUH_API_USERNAME": "wazuh",
10        "WAZUH_API_PASSWORD": " ",
11        "WAZUH_INDEXER_HOST": "10.2.26.23",
12        "WAZUH_INDEXER_PORT": "9200",
13        "WAZUH_INDEXER_USERNAME": "admin",
14        "WAZUH_INDEXER_PASSWORD": " ",
15        "WAZUH_VERIFY_SSL": "false",
16        "WAZUH_TEST_PROTOCOL": "https",
17        "RUST_LOG": "info"
18      }
19    }
20  }
21 }
```

圖十三、mcp.json 的設定

在系統設計過程中，我為 SOC Copilot 設計了一組 system prompt，作為大語言模型在 Wazuh SIEM 與 MCP 工具架構下的行為規格。此 prompt 明確定義模型的角色定位與責任邊界，將其限制為分析輔助者，而非決策或回應執行者，所有輸出僅能基於 MCP 工具回傳的可驗證資料。

在初期設計中，嘗試透過較完整的規則描述與範例 ( few-shot ) 引導模型在不同使用情境下切換回應模式，例如於威脅獵捕情境中進入 Investigation Mode 並套用結構化回應規則，而在系統狀態查詢或端點健康度統計時，僅回傳事實性資訊。同時，prompt 亦支援依使用者輸入自動切換中英文輸出。

然而實際測試後發現，過於細緻的規則與模式切換設計容易使小型語言模型產生過度推理行為，不僅增加回應延遲，也提高在模糊情境下自行補齊敘事、進

而產生幻覺 ( hallucination ) 的風險。

因此，prompt 設計策略調整為採用「Hard Gating + Explicit Non-Inference Defaults」的方式。其中，Hard Gating 用以限制僅在使用者輸入包含特定調查關鍵字時，模型才允許進行威脅分析；其餘情境一律視為查詢或描述性任務。Explicit Non-Inference Defaults 則將「不推理、不下結論、僅陳述事實」設定為模型的預設行為，而非例外狀態。

透過此設計，模型的行為被有效收斂至單一且可預期的回應路徑，在保留必要分析能力的同時，大幅降低過度推理與上下文污染的風險，並明顯提升回應效率與整體穩定性。完整的 prompt 設計內容詳見 system\_prompt.txt。

然後我使用千問三 80 億參數的模型(qwen3-8b)，我有用過其他大語言模型測試過，如 ibm 的 granite 4 H tiny-7b、meta llama 3.1 8B instruct、

Deepseek R1 0528 Qwen3 8B，granite 4 H tiny-7B 和 llama 3.1 8B instruct

沒有雖然都有被訓練可以使用一些工具，但是有時會陷入一直叫 tool 的死循環，而 R1 沒有設計可以用來使用工具，使用者輸入之後會陷入幻覺輸出一些

大語言模型自以為可以用的建議給使用者，所以我最終使用 qwen3-8b。並且

LLM 的溫度(temperature)我設定在 0.4。

然後在 conversation.json 是我在 lmstudio 中對 LLM 做對話的歷史紀錄，其

中我問了六個問題，分別是

1. Analyze recent high-severity Wazuh alerts to assess the current security posture.
2. Investigate agent 003 by correlating its recent alerts, running processes, and open ports.
3. Correlate firewall-related alerts with endpoint authentication failures within a 10-minute window.
4. Based on the available evidence, summarize the incident and suggest the next MCP tool to use.
5. 列出目前所有 Wazuh agent 的狀態。
6. 列出 agent 001 中 vulnerability severity 為 High 的前五個漏洞。

最後得到的結果我非常滿意，不過要注意的是，我雖然在提示詞上下很多功夫，但是 qwen-8b 還是會看過去歷史談話內容，所以我測試後發現重開一個新的對話會有助於切換不一樣的情境時遇到 LLM 幻覺。詳見 1.json、2.json、3.json

## 第二節、將 pfSense 的資料有效傳送到 wazuh 上進行偵測與回應

經過環境設定後，我們可以用 kali 使用 nmap(見圖十四)即可在 opnsense 和 wazuh 收到警告，並且我們可以在 opnsense 上面設定是否要阻擋 IDS 偵測到

的該流量(作法詳見圖十一)。並且我們能分別在 opnsense 和 wazuh 上面收到警告，詳見下圖十五和圖十六。

```
(nisra@nisra)-[~]
$ nmap -sn 10.100.2.2
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-28 14:51 CST
Nmap scan report for 10.100.2.2
Host is up (0.00048s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

圖十四、kali 對端點 windows 做情報蒐集(activate information gathering)

Services: Intrusion Detection: Administration

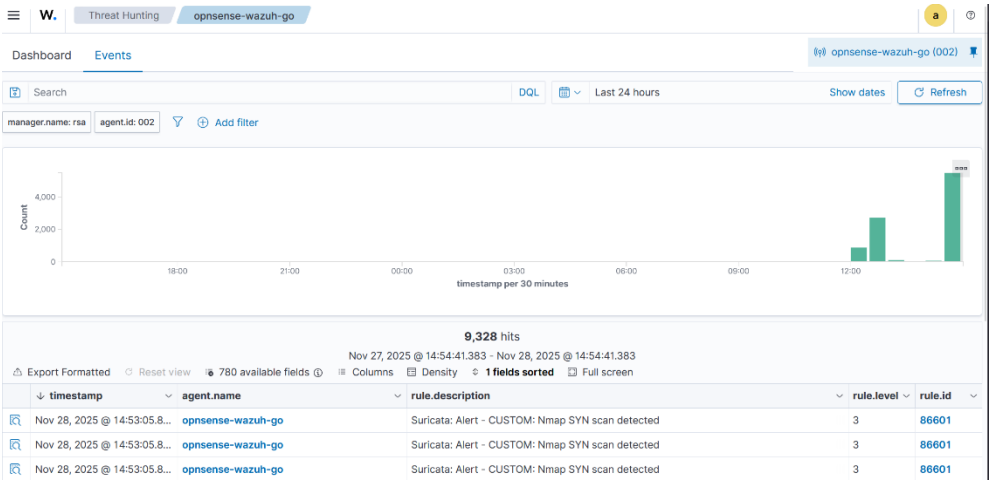
Settings Download Rules User defined Alerts Schedule

Search [icon] [icon] 2025/11/28 14:51 50 [icon]

| Timestamp                       | SID    | Action  | Interface | Source     | Port  | Destination | Port | Alert                          | Info   |
|---------------------------------|--------|---------|-----------|------------|-------|-------------|------|--------------------------------|--------|
| 2025-11-28T14:51:54.995196+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:54.994895+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:54.471927+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:54.471697+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:53.952945+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:53.952252+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:53.439695+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:53.439454+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:52.923665+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:52.923316+0800 | 900001 | allowed | VPN       | 10.100.2.1 | 41572 | 10.100.2.2  | 443  | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:47.499704+0800 | 900001 | allowed | WAN       | 10.2.26.24 | 3324  | 10.2.26.23  | 1514 | CUSTOM: Nmap SYN scan detected | [icon] |
| 2025-11-28T14:51:47.499345+0800 | 900001 | allowed | VPN       | 10.100.2.2 | 50965 | 10.2.26.23  | 1514 | CUSTOM: Nmap SYN scan detected | [icon] |

Showing 1 to 12 of 12 entries

圖十五、opnsense 收到警告





## 圖十六、wazuh 收到警告

### 第三節、ET-BERT 使用體驗

使用 ET-BERT 前，我們先需要把 ET-BERT 整個 repo 從 github 下載後先整理一下，包含，把 dataset 要 fine-tuning 的資料(cstnet-tls1.3 的公開封包數據集)去下載，下載相關依賴的函示庫(對應的 torch)，把 run\_classifier.py 複製一份到 run\_classifier\_infer.py 等。

接下來，根據其 github 的說明，需要先對 pre-trained\_model.bin 進行 fine-tuning(不然最後出來的結果很差，我已經試過了 www)，我使用圖十七的指令去實作，然後再把做完 fine-tuning 的模型(finetuned\_model.bin)用圖十八的指令去做推理(inference)，最後在資料集下有 prediction.tsv 顯示成果。

我稍微比對 test\_dataset.tsv 和模型預測的結果(prediction.tsv)高度吻合，表示我們成功的對 ET-BERT 做 fine-tuning 並且讓模型能夠有效補捉加密流量的行為特徵，而非隨機分類。

```
(etbert) PS D:\ET-BERT> python fine-tuning/run_classifier.py `
>> --pretrained_model_path models/pre-trained_model.bin `
>> --vocab_path models/encryptd_vocab.txt `
>> --train_path datasets/cstnet-tls1.3/packet/train_dataset.tsv `
>> --dev_path datasets/cstnet-tls1.3/packet/valid_dataset.tsv `
>> --test_path datasets/cstnet-tls1.3/packet/test_dataset.tsv `
>> --epochs_num 10 --batch_size 32 --embedding word_pos_seg `
>> --encoder transformer --mask fully_visible `
>> --seq_length 128 --learning_rate 2e-5
```

圖十七、對 pre-trained\_model.bin 進行 fine-tuning 的指令

```
(etbert) PS D:\ET-BERT> python inference/run_classifier_infer.py `
>> --load_model_path models/finetuned_model.bin `
>> --vocab_path models/encryptd_vocab.txt `
>> --test_path datasets/cstnet-tls1.3/packet/nolabel_test_dataset.tsv `
>> --prediction_path datasets/cstnet-tls1.3/packet/prediction.tsv `
>> --labels_num 120 `
>> --embedding word_pos_seg --encoder transformer --mask fully_visible
The number of prediction instances: 58171
```

圖十八、對 finetuned.bin 進行 inference 測試用到的指令

### 第三章、總結及參考資料

#### 第一節、總結

從這次我們作業中，我首先將所有的服務分散不同的設備上，降低單一設備的運行壓力。接著我承接期中報告，詳細設定承擔路由、防火牆、入侵防護於一身的 opnsense 並且將其報告與 wazuh 可以做連動，並且入侵防護方面，做到了自定義規則、上傳規則最後自由決定規則執行的實作。

第一個實作，我在提示詞上下很多功夫，同時也測試很多不一樣的大語言模型，發現有思維鍊的大語言模型會對於第一個時做有更大的幫助。我所設計的實作整體下來使用非常低成本的設備，達到符合預期的樣子，當然如果使用 n8n AI agent workflow 我覺得也會是一個非常有趣的方向。

在第二個實作中，我實際對目標端點 windows 做 activate information gathering 並且演示 opnsense 能夠將辨識到的流量進行警告(alert)或是擋住

(drop)回應的可能性。這個實作我使用了 opnsense 比 pfsense 更加多的 GUI 設計，對於一般的使用者來說，會更加容易使用。

在第三個實作中，我使用 ET-BERT 的預訓練模型為基礎，使用 CSTNET-TLS

1.3 資料集做微調(fine-tuning)，這麼做可以更貼近特定加密流量場景的行為分析，提升分類結果之語意一致性。並且將微調後的模型拿去使用未標註資料進行推論分析，得到非常好的結果，經過微調後的模型可以穩定的捕捉加密流量中特定行為模式。

## 第二節、參考資料

- 期中報告 Threat Hunting Essential - Attack Recognition Techniques, 劉昱甫, 2025.10
- [Tutorial] Adding custom rules to Intrusion Detection, dcol, 2018.02.08, <https://forum.opnsense.org/index.php?topic=7209.0>
- ET-BERT: A Contextualized Datagram Representation with Pre-training Transformers for Encrypted Traffic Classification, Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, Jing Yu, 2022.02.13 ,<https://doi.org/10.48550/arXiv.2202.06335>